devops engineer - detailed guide

DevOps Engineer — Detailed Guide

For someone doing the role. Concepts tied to the actual files in this repository. Read `easy-guide.md` first if any term here is unfamiliar.

Working assets: `infra/` , `docker-compose.yml` , `docker-compose.inference.yml` , `backend/Dockerfile` , `tasks.py` , `Makefile` , `.github/workflows/ci.yml` , `infra/scripts/aws_bootstrap.sh` , `infra/scripts/remote_deploy.sh` , `infra/compose/docker-compose.deploy.yml`

This guide explains *why* the infra is shaped this way, tied to exact files and the real incidents found running it. It intentionally does not restate command-by-command operating procedure — that lives in `infra/docs/` , which is operational documentation, not study material.

---

1. Scope: what “the deployable unit” actually is

```
backend/                    the FastAPI service
frontend/                   the React app + nginx
backend/data/               the 130 MB generated data layer (NOT in git)
docker-compose.yml          local build-and-run stack definition
infra/compose/*.prod.yml    production hardening overlay (local path)
infra/compose/docker-compose.deploy.yml   the real EC2 deploy target (pulls images)
```

`research/` — the model training tree — is **not part of a deployment**. This is the load-bearing architectural decision in this whole project: the API was verified to run correctly against a completely empty `research/` directory. Every route responds — the frozen 28-day forecast, hierarchy, accuracy, insights, `/docs`, every `/genai` route — and `/inference/status` correctly reports itself unavailable with a reason, rather than crashing.

The one exception, live model re-verification (`--inference`), is opt-in specifically so the *default* deployment carries zero dependency on the research tree.

---

2. The data-layer contract (`backend/app/config.py`)

`Settings.required_artefacts` (`config.py:175-180`) names the three files the API needs at `NPN_DATA_DIR`:

```
product.duckdb      20 MB
history.parquet     32 MB
backtest.parquet    78 MB
```

These are **generated**, by `backend/scripts/build_product_db.py`, from frozen research artefacts. They are gitignored and always rebuildable — so they never need backing up. They must simply *exist* before the API can report itself ready.

This single fact drives four different pieces of infra:

Consequence	Where it's enforced
A pre-deploy check must confirm the files exist and are the right size	`infra/scripts/preflight.py` → `check_data_layer()`
A readiness probe must prove the *data*, not just the *process*, is up	`/api/v1/ready` vs. `/api/v1/health` — see §4
A post-deploy check must prove the *served numbers* are the *validated* ones	`infra/scripts/smoke_test.py` — the `3331.3681` canary
The compose file must mount it, read-only	`volumes:` on the `api` service in every compose flavour

On the real EC2 box, this file never travels via `git push` — it's too large and CI has no access to the research tree it's built from. It was shipped **once** through a private S3 bucket (`npn-hackathon-data-<account-id>`, created by `aws_bootstrap.sh`) directly onto the instance at `/opt/npn/backend/data/`, via `aws s3 sync`. That bucket has all public access blocked and is readable only by the EC2 instance's own IAM role — see §7.

---

3. The two-target Dockerfile (`backend/Dockerfile`)

One Dockerfile, two build targets, sharing a base and a dependency layer structure:

```
base ──► deps-api  ──► api    (default; no ML libs, no research mount needed)
     └─► deps-full ──► full   (+ lightgbm/numpy/pandas/pyarrow; opt-in)
```

Non-root from the start. `useradd --uid 10001 ... app` runs before anything else in `base`, and both targets end with `USER app`. CI's `images` job asserts this directly: `docker run --entrypoint id` must report uid **10001**.

Healthcheck uses `/ready`, not `/health` — deliberately, matching §4.

A real, previously-shipped bug: `deps-api`'s dependency-install line read `pip install --require-hashes=false -r requirements.txt`. `pip`'s `--require-hashes` is a boolean flag — it takes no value — so `=false` is invalid syntax and `pip` refuses to run with **exit code 2**. This sat undiscovered because **no image had ever actually been built anywhere**: Docker was not installed on the machine this Dockerfile was written on. The first real CI run against this repository caught it immediately. Fixed by simply dropping the flag: `pip install -r requirements.txt`. The general lesson — a Dockerfile that has never been built is unverified, regardless of how carefully it reads — recurs throughout this whole infra history; see §7.1 for three more instances of exactly this pattern.

---

4. Liveness vs. readiness — where it's actually wired

Probe	Endpoint	Proves	Set as
Liveness	`/api/v1/health`	the process is alive	`livenessProbe`
Readiness	`/api/v1/ready`	the data layer is **queryable**	`readinessProbe`

Both `docker-compose.yml`'s `healthcheck:` block and the Dockerfile's own `HEALTHCHECK` instruction point at `/ready`, and `frontend` depends on `api`'s `service_healthy` condition. That dependency chain is *exactly* why a missing data layer looks like a hang rather than a clear error: the API container never reaches `(healthy)`, so the frontend container never starts at all.

---

5. `docker-compose.yml` vs. `docker-compose.deploy.yml`

The base compose file **builds** images locally (`build: context: .`). The deploy file (`infra/compose/docker-compose.deploy.yml`) **only ever pulls** pre-built images from ECR — the EC2 host has no source tree, no Dockerfile, and no build toolchain on it at all:

```
services:
  api:
    image: ${ECR_REGISTRY}/npn-api:${IMAGE_TAG:-latest}
    ...
    volumes:
      - /opt/npn/backend/data:/data/product:ro
  frontend:
    image: ${ECR_REGISTRY}/npn-frontend:${IMAGE_TAG:-latest}
    ports:
      - "8080:8080"
```

It folds the same production hardening the local `infra/compose/docker- compose.prod.yml` overlay provides — `read_only: true`, `cap_drop: ALL`, resource limits, log rotation — directly into one self-contained file, since the deploy host has nothing to layer it on top of.

A real bug lived here, found only once the container actually ran on the real box. `frontend`'s `tmpfs:` list covered `/var/cache/nginx`, `/var/run`, and `/tmp` — the three paths nginx's own documentation says it needs. It did **not** cover `/etc/nginx/conf.d`, which is exactly where the image's entrypoint renders `/etc/nginx/templates/*.template` via `envsubst` on every single boot (see `frontend/Dockerfile`'s final `CMD`). Under `read_only: true` with no tmpfs there, the container didn't run insecurely — it didn't run **at all**: `docker logs` showed `Read-only file system` and an immediate crash-loop, confirmed live via `docker compose -f docker-compose.deploy.yml ps` showing `Exited (1)`. Fixed with one more `tmpfs` entry:

```
- /etc/nginx/conf.d:size=8m
```

The identical gap existed in the older, never-actually-run-under-Docker `infra/compose/docker-compose.prod.yml` overlay too, and was fixed there at the same time — its own `handover.md` had explicitly flagged this file's `--prod` tmpfs behaviour as "reasoned from the image's behaviour, not tested," which is exactly the class of risk that materialised.

---

6. `nginx.conf` — the header-inheritance bug

`frontend/nginx.conf` sets three security headers at the `server` block level (`X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`), intending them to apply to every response.

nginx does not inherit `add_header` from a parent context into a child `location` block that defines even one `add_header` of its own — this is documented nginx behaviour, not a bug in nginx, but it is exactly the kind of gotcha that only surfaces by inspecting real response headers. Two `location` blocks each set their own `Cache-Control` header:

```
location /assets/ {
    add_header Cache-Control "public, immutable";   # <- this line alone
}                                                     #    silently drops
```

```
                                                    #    the parent's
location = /index.html {                            #    three headers
    add_header Cache-Control "no-cache";
}
```

Because `try_files ... /index.html` internally redirects almost every real page load to the exact-match `location = /index.html` block, **almost every real request lost the three security headers** — `curl -sI` against the live deployment confirmed `Cache-Control` present and all three security headers absent. `infra/scripts/smoke_test.py` had been reporting this as a `[ warn ]`, not a `[ fail ]` — worth noticing warnings, not just failures. Fixed by repeating all three headers explicitly inside both child `location` blocks — the only correct fix per nginx's own documented (non-)inheritance model; there is no "make it inherit" flag.

---

7. AWS bootstrap (`infra/scripts/aws_bootstrap.sh`)

One idempotent script, plain AWS CLI (matching this project's existing style of `tasks.py` / `preflight.py` — a script you can read top to bottom, not a state file you have to trust), that provisions everything except the EC2 instance itself:

Resource	Purpose
ECR repos `npn-api`, `npn-frontend`	image registry
S3 bucket `<project>-data-<account>`	one-time data-layer transfer + compose-file handoff at deploy time
GitHub OIDC identity provider	lets GitHub Actions prove its identity to AWS with no stored password
IAM role `<project>-github-deploy`	what GitHub Actions assumes — scoped to *this repo, this branch* only
IAM role + instance profile `<project>-ec2-*`	what the EC2 host itself runs as — SSM + ECR-pull + scoped S3/SSM-parameter read, nothing broader
Security group	inbound `tcp/8080` only — **no port 22 at all**; the deploy path uses SSM, never SSH
SSM parameter `/<project>/gemini-api-key`	the one real secret, `SecureString`, placeholder value `"unset"` until a real key is set

Re-running the script is safe — every resource is looked up before being created or updated, so a partial prior run never produces duplicates.

7.1 Three more real bugs, all found only by actually running the deploy

Continuing the pattern from §3 — every one of these looked correct on paper and failed the first time it actually ran against real AWS.

Bug A — a stale OIDC thumbprint. The script originally hard-coded `--thumbprint-list "6938fd4d98bab03faadb97b34396831e3780aea1"` when creating the GitHub OIDC provider — a widely copy-pasted value from older tutorials, assuming GitHub's token-signing certificate still chained through the same root CA it did when that value was current. It doesn't anymore — GitHub's current chain roots through Let's Encrypt. AWS's error for this, `Not authorized to perform sts:AssumeRoleWithWebIdentity`, is **identical** to the error for a wrong trust-policy condition, giving no hint that the thumbprint specifically is stale. Diagnosed by fetching the live certificate chain directly and computing its real SHA-1 fingerprint:

```
openssl s_client -servername token.actions.githubusercontent.com \
  -connect token.actions.githubusercontent.com:443 -showcerts </dev/null \
  | openssl x509 -noout -fingerprint -sha1
```

Fixed by computing this live inside the script itself, every run, instead of trusting a hard-coded value — and by making an existing provider's thumbprint self-heal via `update-open-id-connect-provider-thumbprint` on every re-run, not just at creation time (`aws_bootstrap.sh`'s `OIDC_ARN` block).

Bug B — the trust policy didn't match GitHub's real token shape. Even with the thumbprint fixed, the same generic `Not authorized` error persisted. The textbook OIDC `sub` claim format is `repo:owner/repo:ref:refs/heads/main`, and the trust policy matched exactly that. The *actual* claim GitHub sent — found by reading the raw failed API call in **AWS CloudTrail**, not by reading documentation harder — was:

```
repo:iwinalbert@127975274/npn-hackathon@1338173409:ref:refs/heads/main
```

GitHub appends immutable numeric owner/repo IDs to the `sub` claim as protection against a renamed account or repository silently inheriting an old trust relationship. Confirmed against `gh api repos/.../npn-hackathon` (`owner.id` and `id` matched exactly). Fixed with a `StringLike` condition matching **both** shapes, so it survives either format GitHub sends:

```
"token.actions.githubusercontent.com:sub": [
  "repo:iwinalbert/npn-hackathon:ref:refs/heads/main",
  "repo:iwinalbert@*/npn-hackathon@*:ref:refs/heads/main"
]
```

Bug C — `docker images` doesn't take two repository arguments. In `ci.yml`'s `images` job, `docker images --format '...' npn-forecast-api npn-forecast-frontend` failed with `'docker images' requires at most 1 argument`. Never caught because this step had never executed on a machine with Docker installed. Fixed with two `--filter reference=` flags instead, which Docker OR's together:

```
docker images --format '...' \
  --filter reference='npn-forecast-api' \
  --filter reference='npn-forecast-frontend'
```

The pattern across all seven bugs in this document (§3, §5, §6, and these three) is the same one stated in the easy guide: nothing here was caught by reading the code, however carefully. Every one was

caught by actually running it — a build, a container boot, a real AWS API call — for the first time.

---

8. `remote_deploy.sh` and the SSM deploy path

`infra/scripts/remote_deploy.sh` is what the `deploy` CI job actually runs **on the box**, via `aws ssm send-command` with `AWS-RunShellScript`. It is deliberately a plain, standalone script rather than inline YAML in `ci.yml`, because an SSM command body with the level of quoting a multi-step deploy needs is unreadable and effectively untestable inline. Four steps:

```
aws ecr get-login-password ... | docker login ...
GEMINI_API_KEY=$(aws ssm get-parameter --name "$GEMINI_PARAM_NAME" \
  --with-decryption --query Parameter.Value --output text)
[ "$GEMINI_API_KEY" = "unset" ] && GEMINI_API_KEY=""
docker compose -f docker-compose.deploy.yml up -d
docker image prune -f
```

The `"unset"` check exists because that literal string is the placeholder value `aws_bootstrap.sh` writes when no real key has been configured yet — passing it through unchanged would make the API try to authenticate to Gemini with the string `"unset"` as a key, rather than correctly reporting the assistant as unavailable.

Deploy is polled, not fire-and-forget. The `deploy` job in `ci.yml` sends the SSM command, then polls `aws ssm get-command-invocation` in a loop (up to 30 × 6s) until it reports `Success`, `Failed`, `Cancelled`, or `TimedOut` — printing the command's real stdout/stderr either way. A CI job that just fires the SSM command and reports "done" would be lying about what it actually verified.

The very last step of the `deploy` job re-runs `infra/scripts/smoke_test.py` against the box's real public IP — the same canary check from §10 of the easy guide, now run automatically, from GitHub's own infrastructure, against the real deployed system, on every push.

---

9. Pausing the deployment (`aws ec2 stop-instances`)

Stopping the EC2 instance halts compute billing without touching anything else: the EBS volume (with the data layer already synced onto it), the Elastic IP allocation, the ECR images, the IAM roles, the S3 bucket, and the SSM parameter all persist untouched. Starting it again (`start-instances`) restores the exact same box.

One consequence worth knowing operationally: if code is pushed to `main` while the instance is stopped, the `publish` job still succeeds (it only talks to ECR), but the `deploy` job's SSM command will fail to reach a stopped instance — this is expected, harmless, and does not touch the instance in any way. It shows as a red `X` in the Actions tab and nothing more. There is no need to "clean up" after this; the next push after the instance is restarted deploys normally.

10. CI (`ci.yml`) — what changed, and why

The backend and frontend jobs no longer run a test suite. Earlier in this project's history, `backend/tests/` (13 files, 149 tests — including a freeze-regression guard that hashed the frozen model binaries and asserted served forecasts matched the frozen CSV row-for-row) and `frontend/src/test/` (6 files, 65 tests) existed and ran on every push. Both directories were later deleted from the repository entirely, along with:

- the CI steps that ran them (`backend`'s `Run tests` + `Summarise`, `frontend`'s `Test`);
- `backend/pytest.ini`'s `testpaths` (pointed at the now-deleted directory) and its `slow` / `live` markers (meaningless with nothing to mark);
- the `pytest` / `httpx` entries in `backend/requirements-dev.txt` (test-only; nothing in `app/` or `scripts/` imports either);
- `frontend/vite.config.ts`'s `test:` block (referenced the deleted `src/test/setup.ts`);
- `frontend/tsconfig.json`'s `vitest/globals` and `@testing-library/jest-dom` type references (no longer resolvable);
- the `vitest` / `@testing-library/*` / `jsdom` devDependencies in `frontend/package.json`, with `package-lock.json` regenerated to match.

What this means concretely for what CI proves today: the `backend` job verifies the app *imports* cleanly and the committed `openapi.json` is current; the `frontend` job verifies the app *typechecks* and *builds*; the `images` job still builds real containers, boots them, and asserts they degrade correctly with no data mount. None of that is nothing — a broken import, a stale schema, a failed typecheck, or a container that won't boot are all still caught. But **there is no longer an automated check that application *logic* is correct** — no assertion that a given store/item still returns the right forecast shape, no check that the GenAI guardrails still refuse what they're supposed to refuse, no regression guard on the frozen model's hashes. If you are asked to verify a change is *behaviourally* correct, not just that it builds, that verification currently has to happen by hand, since the automation that used to do it no longer exists in this repository.

11. Where to go next

- `infra/docs/environment-reference.md` — every environment variable, source-of-truth is `backend/app/config.py`.
- `infra/docs/deployment-guide.md` — the decisions you have to make deploying somewhere new.
- `infra/docs/runbook.md` — day-to-day operation: rollback, key rotation, incident response.

- `infra/docs/troubleshooting.md` — the full symptom → cause → fix reference; skim it once so symptoms are recognisable later.
- `infra/scripts/aws_bootstrap.sh` and `infra/scripts/remote_deploy.sh` — read both top to bottom; each is short enough to hold in your head at once, which is deliberate.